Greedy algorithms solutions

16-E01 Activity selection and exchange

Derivation and check

Select(1,3),then(4,8). It overlaps(7,9), so only two are chosen. Starting with(2,5) can choose(7,9),also two,but no better. Exchange says some optimum can replace its first activity by earliest finish without losing size; it does not require every optimum to contain that activity.

16-E02 Coin-greedy counterexample

Derivation and check

Greedy takes5+5+1+1,four coins. Optimum4+4+4 uses three. This refutes the unrestricted largest-coin rule. A valid proof needs a canonical property for the denomination family; familiar1,5,10,25 cases do not imply it for other families.

16-E03 Huffman merges

Derivation and check

Merge C1+D1=2,then B2+2=4,then A4+4=8. One code is A=0,B=10,C=110,D=111. Weighted length4・1+2・2+1・3+1・3=14. Swapping C,D or tied left/right branches can change the code without changing optimal cost.

16-E04 Weighted activities need DP

Derivation and check

Sort short,long,tail by finish. Let p(j) be the final earlier finish≤start_j,or0; D[0]=0 and D[j]=max(D[j−1],value_j+D[p(j)]). short:p0,D1=1; long:p0,D2=100; tail:p2,D3=101. Optimum{long,tail} has101; greedy{short,tail} has2. A later-finishing activity can carry more value, so cardinality exchange cannot control weighted value. Optimal substructure remains, but include/exclude choices must both be compared.

Greedy algorithms solutions (continued)

16-E05 Augmentation does not follow from heredity

Derivation and check

Every subset of each listed independent set is also listed,including{b},{c},∅ beneath{b,c}; heredity holds. A={a},B={b,c} have1<2,yet neither{a,b} nor{a,c} is independent,violating augmentation. Greedy selects a and rejects b,c for value10,while independent{b,c} has18. In a matroid some optimum would contain first feasible a; this system lacks the required augmentation. Empty-set membership and heredity alone are insufficient. Include independence-oracle cost in time bounds.

16-E06 Deadline capacities and a full schedule

Derivation and check

Order r,s,t,u,v. Accept r with prefixes(1,1,1),then s with(1,2,2). Trying t gives(2,3,3),violates N1≤1,and is rejected,restoring(1,2,2). Accept u with(1,2,3). Trying v gives(1,3,4),violates N2≤2,and is rejected. Schedule r,s,u at1,2,3 and t,v at4,5. Minimum penalty10+7=17.

For general augmentation,set k=max{t:N_t(A)=t},including0. If|A|<|B|,A has|A|−k jobs with deadline>k,B at least|B|−k,so choose x in B outside A with deadline>k. For t≤k,adding it changes nothing; for t>k,N_t(A)≤t−1,so adding at most1 remains feasible. With heredity and empty-set feasibility,this is a matroid and positive-penalty greedy is globally optimal. k is the last tight capacity of A,not the last equality between A and B prefixes; prefix differences can cross zero without attaining zero.

16-E07 Huffman exchange sign and singleton boundary

Derivation and check

Before the swap,the two leaves cost1・1+5・3=16; afterward1・3+5・1=8. New minus old is−8. Generally f_x d_a+f_a d_x−f_x d_x−f_a d_a=(f_a−f_x)(d_x−d_a)≤0 because f_a≥f_x,d_x≤d_a. The opposite depth difference gives old minus new and cannot be labeled new minus old.

A singleton has no merges and tree depth0,so theoretical weighted cost0. Prescribing code0 for f_x occurrences costs f_x bits and requires separately communicated message length. Merge-weight sum equals weighted tree depth,but excludes the implementation’ s forced extra bit per singleton occurrence. Keep these cost models distinct.

Greedy algorithms solutions (continued)

Verification code appendix: greedy_revision.py

```
"""Exact traces and exchange checks for the Chapter 16 revision pages."""

from dataclasses import dataclass
from heapq import heapify, heappop, heappush
from itertools import combinations, product
from functools import lru_cache


@dataclass(frozen=True)
class Activity:
    name: str
    start: int
    finish: int
    value: int = 1


def activity_selection(activities):
    ordered = sorted(activities, key=lambda activity: (activity.finish, activity.start))
    if not ordered:
        return [], []
    chosen = [ordered[0]]
    rejected = []
    for activity in ordered[1:]:
        if activity.start >= chosen[-1].finish:
            chosen.append(activity)
        else:
            rejected.append(activity)
    return chosen, rejected


def weighted_activity_dp(activities):
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy algorithms solutions (continued)

```
    ordered = sorted(activities, key=lambda activity: activity.finish)
    end_times = [activity.finish for activity in ordered]
    previous = []
    for current, activity in enumerate(ordered):
        compatible = [
            index
            for index, earlier in enumerate(ordered[:current])
            if earlier.finish <= activity.start
        ]
        previous.append(max(compatible) if compatible else -1)

    best = [0] * (len(ordered) + 1)
    choice = [False] * (len(ordered) + 1)
    for count in range(1, len(ordered) + 1):
        include = ordered[count - 1].value + best[previous[count - 1] + 1]
        exclude = best[count - 1]
        if include > exclude:
            best[count] = include
            choice[count] = True
        else:
            best[count] = exclude

    selected = []
    count = len(ordered)
    while count:
        if choice[count]:
            selected.append(ordered[count - 1])
            count = previous[count - 1] + 1
        else:
            count -= 1
    return best[-1], tuple(reversed(selected)), tuple(end_times)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy algorithms solutions (continued)

```
def huffman(frequencies):
    heap = [(weight, serial, name) for serial, (name, weight) in
enumerate(frequencies.items())]
    heapify(heap)
    merges = []
    serial = len(heap)
    while len(heap) > 1:
        left_weight, _, left_tree = heappop(heap)
        right_weight, _, right_tree = heappop(heap)
        weight = left_weight + right_weight
        merges.append((left_tree, right_tree, weight))
        heappush(heap, (weight, serial, (left_tree, right_tree)))
        serial += 1
    tree = heap[0][2]

    codes = {}

    def collect(node, prefix):
        if isinstance(node, str):
            codes[node] = prefix or "0"
            return
        collect(node[0], prefix + "0")
        collect(node[1], prefix + "1")

    collect(tree, "")
    cost = sum(frequencies[name] * len(code) for name, code in codes.items())
    return tree, merges, codes, cost


def independent_tasks(tasks):
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy algorithms solutions (continued)

```
    return all(
        deadline >= position
        for position, deadline in enumerate(sorted(task[1] for task in tasks), 1)
    )


def task_schedule(tasks):
    selected = []
    rejected = []
    for task in sorted(tasks, key=lambda item: (-item[2], item[1], item[0])):
        if independent_tasks(selected + [task]):
            selected.append(task)
        else:
            rejected.append(task)
    early_order = sorted(selected, key=lambda item: (item[1], item[0]))
    late_penalty = sum(item[2] for item in rejected)
    return selected, rejected, early_order, late_penalty


def hereditary_greedy_counterexample():
    elements = {"a": 10, "b": 9, "c": 9}
    independent = {
        frozenset(),
        frozenset({"a"}),
        frozenset({"b"}),
        frozenset({"c"}),
        frozenset({"b", "c"}),
    }
    assert all(
        subset in independent
        for item in independent
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy algorithms solutions (continued)

```
        for size in range(len(item) + 1)
        for subset in map(frozenset, combinations(item, size))
    )
    greedy = []
    for item, _ in sorted(elements.items(), key=lambda pair: -pair[1]):
        if frozenset(greedy + [item]) in independent:
            greedy.append(item)
    assert greedy == ["a"]
    assert elements["b"] + elements["c"] == 18 > elements["a"]
    return greedy, (elements["b"] + elements["c"])


@lru_cache(None)
def exhaustive_merge_cost(weights):
    if len(weights) < 2:
        return 0
    return min(
        weights[i]+weights[j]+exhaustive_merge_cost(tuple(sorted(
            [weight for k, weight in enumerate(weights) if k not in (i,j)]
            + [weights[i]+weights[j]]
        )))
        for i in range(len(weights)) for j in range(i+1,len(weights))
    )


def audit_exchange_and_schedules():
    assert activity_selection(()) == ([], [])
    old_cost, new_cost = 1*1+5*3, 1*3+5*1
    delta = (5-1)*(1-3)
    assert new_cost-old_cost == delta == -8
    assert new_cost-old_cost != (5-1)*(3-1)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy algorithms solutions (continued)

```
    merge_cases = 0
    for n in range(2, 6):
        for weights in product(range(1,4),repeat=n):
            _, merges, _, cost = huffman(dict(zip("ABCDE",weights)))
            assert cost == sum(merge[2] for merge in merges)
            assert cost == exhaustive_merge_cost(tuple(sorted(weights)))
            merge_cases += 1

    augmentation_cases = 0
    for n in range(1,5):
        for deadlines in product(range(1,n+1),repeat=n):
            tasks = tuple((str(i), d, n-i) for i,d in enumerate(deadlines))
            feasible = [set(ids) for k in range(n+1)
                        for ids in combinations(range(n),k)
                        if independent_tasks([tasks[i] for i in ids])]
            selected, _, _, _ = task_schedule(tasks)
            optimum=max(sum(tasks[i][2] for i in ids) for ids in feasible)
            assert sum(task[2] for task in selected)==optimum
            for a in feasible:
                for b in feasible:
                    if len(a)>=len(b):
                        continue
                    tight=max(t for t in range(n+1)
                              if sum(tasks[i][1]<=t for i in a)==t)
                    options=[i for i in b-a if tasks[i][1]>tight]
                    assert options
                    assert all(independent_tasks([tasks[i] for i in a|{x}]) for x in options)
                    augmentation_cases += 1
    print(f"PASS negative_exchange_delta={delta} exhaustive_huffman={merge_cases}
deadline_augmentation_pairs={augmentation_cases}")
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Greedy algorithms solutions (continued)

```
def main():
    activities = (
        Activity("a", 1, 3),
        Activity("b", 2, 5),
        Activity("c", 3, 6),
        Activity("d", 5, 7),
        Activity("e", 7, 9),
        Activity("f", 8, 10),
    )
    chosen, rejected = activity_selection(activities)
    assert [item.name for item in chosen] == ["a", "c", "e"]
    assert [item.name for item in rejected] == ["b", "d", "f"]

    weighted = (
        Activity("short", 0, 2, 1),
        Activity("long", 1, 3, 100),
        Activity("tail", 3, 4, 1),
    )
    earliest_finish = [weighted[0], weighted[2]]
    best_value, best_set, _ = weighted_activity_dp(weighted)
    assert sum(item.value for item in earliest_finish) == 2
    assert best_value == 101 and [item.name for item in best_set] == ["long", "tail"]

    frequencies = {"A": 5, "B": 2, "C": 1, "D": 3}
    tree, merges, codes, huffman_cost = huffman(frequencies)
    assert [merge[2] for merge in merges] == [3, 6, 11]
    assert huffman_cost == 20

    tasks = (
        ("a", 3, 9),
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy algorithms solutions (continued)

```
        ("b", 1, 8),
        ("c", 3, 7),
        ("d", 2, 6),
        ("e", 1, 5),
    )
    selected, rejected_tasks, early_order, late_penalty = task_schedule(tasks)
    assert [item[0] for item in selected] == ["a", "b", "c"]
    assert [item[0] for item in rejected_tasks] == ["d", "e"]
    assert [item[0] for item in early_order] == ["b", "a", "c"]
    assert late_penalty == 11

    hereditary_greedy, hereditary_best = hereditary_greedy_counterexample()
    assert hereditary_greedy == ["a"] and hereditary_best == 18
    audit_exchange_and_schedules()

    print(
        "PASS "
        f"activity={[item.name for item in chosen]} "
        f"weighted_best={best_value}:{[item.name for item in best_set]} "
        f"huffman_merges={[merge[2] for merge in merges]} "
        f"huffman_codes={codes} cost={huffman_cost} "
        f"task_early={[item[0] for item in early_order]} "
        f"penalty={late_penalty} "
        f"hereditary_greedy={hereditary_greedy}/{hereditary_best}"
    )


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study